1. Objetivo

O objetivo deste trabalho é implementar um mecanismo de sincronização de threads para o sistema de embarque de passageiros em vagões do METROREC. A solução garante que o embarque ocorra de forma ordenada e simultânea, sem busy-wait, utilizando apenas um mutex e duas variáveis de condições por estação.

2. Checklist de Requisitos

Um requisito por linha. Sem parágrafo, se precisar explicar algo, use a coluna "como testar".

Requisito do enunciado	Status	Como testar / evidência
Readme + Makefile	OK	Compilação via Makefile gerando sync_car, com test_sync_car.c na pasta(make clean && make)
Relatório	OK	vgjda2.pdf
Sem semáforos/outras primitivas	OK	Inspeção do código-fonte, apenas pthread_mutex e pthread_cond foram utilizados.
Apenas 1 lock por struct station	OK	Verificação do campo lock único na struct station em sync_car.c.
Sem busy - wait	OK	Laços utilizam pthread_cond_wait enquanto a condição de parada não é atendida.
Embarque simultâneo	OK	Múltiplos passageiros reservam assentos e retornam de station_wait_for_car antes de chamarem station_on_board.

3. Como Reproduzir

Para reproduzir os testes:
1. Copie o arquivo de teste 'test_sync_car.c' para a pasta 'vgjda2'.
Compilar:

```
make clean
make
```

Executar:

```
./sync_car
```

4. Arquitetura

Arquivos e responsabilidades:

1. **sync_car.h → Contém as definições da struct station e os protótipos das quatro funções de sincronização.**
2. **sync_car.c → Contém as funções de sincronização (station_init, station_load_car, station_wait_for_car, station_on_board)**

3. **Makefile → Automação do processo de compilação do código C com a *flag* -pthread e regra de limpeza (clean), ele gera sync_car.o e, havendo um .c de teste na pasta, o executável sync_car, com as regras all, run e clean. .**

Decisões técnicas:

1. **Duas Variáveis de Condição (chegada_carro e passageiro_sentado) → Separar os eventos de chegada do trem e de confirmação de assento ocupado → Evita acordar *threads* desnecessárias. As duas variáveis existem para separar quem espera: passageiros esperam o carro e o carro espera o embarque terminar. Assim o signal do on_board acorda só o carro.**

2. **Controle com Variáveis Auxiliares (esperando, assentos_livre, boarding, carro_presente) → Registrar o estado da estação dentro da seção crítica → Garante consistência na contagem de assentos e impede corridas de dados (*data races*).**

3. **Separação com sync_car.h → Garantir compatibilidade com os testes automatizados da disciplina que exigem #include "sync_car.h". → Estruturação mais limpa do código e facilidade de compilação sem alterar o arquivo principal.**

5. Estratégias e Diário de Desenvolvimento

5.1 Estratégias da semana

Estratégia	Nome curto	Contexto	Motivo da troca (se houve)
S1	Desenvolvimento Incremental com Testes Modulares	Implementação sequencial e isolada de cada função do monitor (station_init, station_wait_for_car, station_load_car e station_on_board) utilizando *mains* temporárias e validações por *stubs* antes da integração final com a suíte automatizada test_sync_car.c.	Não houve

5.2 Diário de Tentativas

#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência
1	S1	Compilar sync_car.c com make dentro da pasta vgjda2	Falha	Erro implicit declaration of function printf por falta de #include <stdio.h> no sync_car.c	31/10/2026 – 06:22	evidencias.log (1 - 65)
2	S1	Incluir #include <stdio.h> no topo do sync_car.c e compilar com make	Sucesso		31/10/2026 – 06:23	evidencias.log(1 - 65)

#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência
3	S1	Compilar o teste de station_wait_for_car com make	Falha	Erro implicit declaration of function sleep por ausência de #include	31/10/2026 - 06:52:56	evidencias.log(66 - 108)
4	S1	Incluir #include <unistd.h> e reexecutar a compilação e teste	Sucesso		31/10/2026 - 06:52: 56 - 06:55:28	evidencias.log(66 - 108)
5	S1	Compilar sync_car.c via make na pasta vgjda2 após adicionar station_load_car	Falha	Erro de digitação ststation e ausência de ponteiro (&) na chamada pthread_mutex_unlock	31/10/2026 – 07:44:51 - 08:10:16	evidencias.log(109- 207)
6	S1	Recompilar sync_car.c via make após corrigir a chamada do mutex	Falha	Erro undefined reference to main ao tentar gerar executável a partir de um arquivo de biblioteca	31/10/2026 – 07:44:51 - 08:10:16	evidencias.log(109- 207)
7	S1	Tentar compilar com a suíte oficial test_sync_car.c do professor	Falha	Erro undefined reference to station_on_board pois a função ainda não tinha sido implementada.	31/10/2026 – 07:44:51 - 08:10:16	evidencias.log(109- 207)
8	S1	Adicionar main temporária com stub no sync_car.c e testar station_load_car sem passageiros	Sucesso		31/10/2026 – 07:44:51 - 08:10:16	evidencias.log(109- 207)
9	S1	Compilar sync_car.c contendo redefinição de station_on_board e sem a cópia do arquivo de testes	Falha	Erro de redefinição de função no .c e falha ao localizar ../test_sync_car.c.	31/10/2026 – 08:33:28 - 08:43:04	evidencias.log(208 - 288)
10	S1	Bug em em station_on_board e test_sync_car(Rodar o teste oficial com station_on_board sem	Falha	O trem entra no station_load_car com 30 assentos, não recebe a notificação dos passageiros sentados e estoura o alarme (Error:	31/10/2026 – 08:33:28 - 08:43:04	evidencias.log(208 - 288)

#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência
		pthread_cond_signal)		station_load_car failed).		
11	S1	Compilar sync_car.c em conjunto com test_sync_car.c e executar a suíte automatizada	Sucesso		31/10/2026 – 09:28:09-03 - 09:29:37	evidencias.log(289 - 327)
12	S1	Teste limites com testes_limites.c	Sucesso		31/10/2026 – 10:35:00 - 10:35:55	evidencias.log(328 -355)

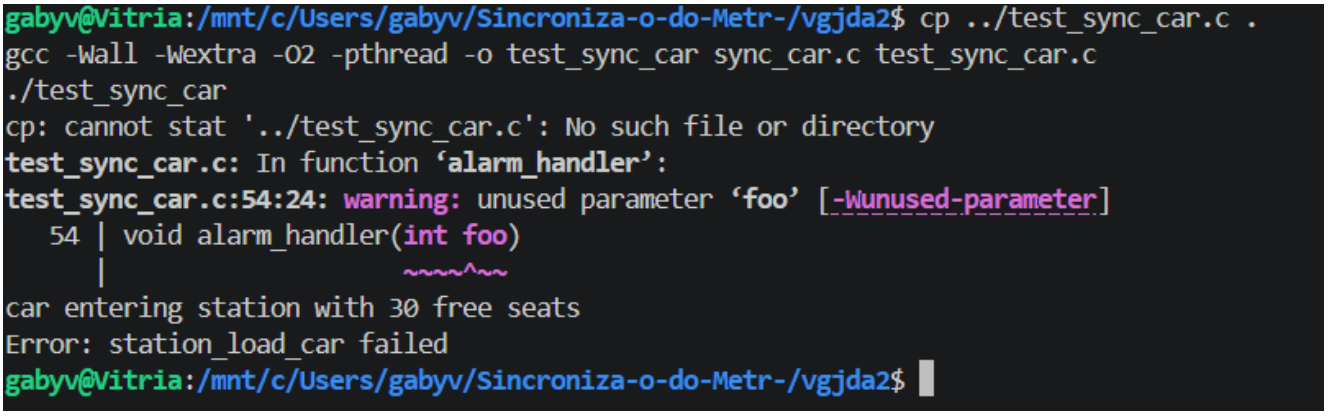
6. Evidências

6.1 Log automático (faz o trabalho pesado por você)

```
script -a evidencias.log
```

O arquivo evidencias.log foi gerado durante o desenvolvimento via terminal WSL com o comando script -a evidencias.log contendo a execução dos comandos date, whoami e pwd, sendo incluído dentro da pasta compactada vgjda2.tar.

6.2 Prints



Legenda: Falha no teste de sincronização (Error: station_load_car failed) ao executar a station_on_board sem a notificação do *mutex* via pthread_cond_signal, resultando no encerramento por tempo limite (*timeout*).

7. Uso de IA

Onde usei IA:

Usei IA para ajudar a estudar o conteúdo necessário para fazer essa implementação, Usei para tirar dúvidas durante a escrita do código e testes com main temporário e o .c dos teste limite, ela também foi utilizada para a revisão de escrita deste relatório.

Prompts principais :

- **Me explique o conteúdo necessário para eu conseguir implementar essa atividade.**
- **Me ajude a testar essa parte do código(a IA me mandava um main e um caso).**
- **Me ajude a revisar esse relatório, indique se tiver algum erro de escrita.**
- **Crie um teste limite para validar o comportamento da estação sob carga ou cenários extremos.**

O que validei manualmente e como:

- **Compilação e Flags do GCC: Rodei o comando make e gcc manualmente no terminal Linux (WSL) verificando se o código compilava sem *warnings* (-Wall -Wextra) e com a biblioteca de threads ativada (-pthread).**
- **Teste Limite e Validação Integrada Final: Executei manualmente o teste de estresse/limite fornecido e a suíte oficial test_sync_car.c, nessa execução foi acompanhado a movimentação de 100 passageiros distribuídos em 5 viagens até a obtenção da confirmação final Looks good!.**

Testes por Função: Criei funções main temporárias para validar cada função individualmente:

- **station_init: Verifiquei via printf se todas as variáveis da struct station eram inicializadas zeradas.**
- **station_wait_for_car: Criei *threads* de passageiros para validar se o passageiro decrementava os assentos disponíveis, incrementava a contagem de *boarding* e saía da fila de espera sem *busy-wait*.**
- **station_load_car: Validei o comportamento da chegada do trem em uma estação sem passageiros aguardando, confirmando via printf que a função atualizou os estados e retornou imediatamente sem entrar em *deadlock* ou *busy-wait*.**
- **station_on_board: Testei a decrementação do contador boarding e a sinalização exata via pthread_cond_signal para liberar o vagão no momento em que o último passageiro senta.**

8. Reflexão Final

A maior dificuldade foi entender que o carro precisa esperar não só passageiros sem assento, mas também os que já reservaram e ainda não chamaram station_on_board. Sem o pthread_cond_signal no on_board, o carro nunca acordava e o teste do professor estourava o alarme (entrada #10). Duas variáveis de condição, chegada_carro e passageiro_sentado, deixam o signal acordar só o carro. Limitação: o broadcast acorda todos os passageiros, e a ordem de embarque não é FIFO.

9. Checklist Final de Entrega

Item	Confirmado?
Compilei do zero seguindo só o que está no relatório	OK
Rodei os testes e evidencias.log foi gerado	OK
Tenho prints obrigatórios	OK
Testei pelo menos um caso limite e um caso inválido	OK
Preenchi a seção de uso de IA	OK
Revisei o relatório e removi frases genéricas / vazias	OK

10. Se eu tivesse mais 2 horas

Se tivesse mais duas horas, criaria um script para testar 1.000 threads de passageiros simultâneas para validar o sistema sob estresse. Além disso, usaria o Valgrind (Helgrind) e o ThreadSanitizer para detectar os *data races* e *deadlocks*. Por fim, adicionaria logs de *debug* condicionais com #ifdef para acompanhar as trocas de estado da estação no terminal.